Python Engineering - Reference Guide

Class 06 - Dictionaries & Sets: guia 'para tontos'

Referencia ampliada para **week_02_data_structures -> class_06_dictionaries_sets**. La idea es poder volver dentro de meses y recordar rapidamente que es un diccionario, que es un set, en que se diferencian, como se accede a sus datos y cuando conviene utilizar cada uno.

1. Diccionario: definicion intuitiva

Un **diccionario** es una coleccion de pares **clave: valor**. En vez de buscar un dato por una posicion numerica, como hacemos en una lista, lo buscamos por un nombre o clave que describe lo que significa.

```
config = {
    'h0': 0.0,
    'v0': 50.0,
    'g': 9.81,
    'dt': 0.1
}

config['v0']   # 50.0
config['dt']   # 0.1
```

Regla mental: lista = 'dame el elemento de la posicion 2'. Diccionario = 'dame el valor asociado a la clave dt'.

2. Caracteristicas principales de un diccionario

Caracteristica	Significado
Pares clave:valor	cada dato se identifica mediante una clave
Claves unicas	una misma clave no puede representar dos entradas distintas
Mutable	puedes anadir, actualizar y eliminar entradas
Acceso por clave	no necesitas recordar indices numericos
Puede mezclar tipos	valores pueden ser numeros, strings, listas, otros dict, etc.
Orden de insercion	Python moderno conserva el orden en que se insertan las claves

3. Crear, leer y modificar un diccionario

```
config = {'h0': 0.0, 'v0': 50.0}

# Read
v0 = config['v0']

# Update existing key
config['v0'] = 60.0

# Add new key
config['dt'] = 0.1

# Delete
del config['h0']
```

4. Acceso seguro: [] frente a .get()

```
config['dt']
# KeyError if 'dt' does not exist

config.get('dt')
# None if it does not exist

config.get('dt', 0.01)
# returns 0.01 if missing
```

`d[key]` es util cuando la clave DEBE existir. `d.get(key)` es util cuando una clave puede faltar y quieres manejarlo sin error.

5. Metodos y funciones mas importantes de dict

Metodo / funcion	Que hace	Ejemplo
d.keys()	devuelve una vista de las claves	config.keys()
d.values()	devuelve una vista de los valores	config.values()
d.items()	pares (clave, valor)	config.items()
d.get(k)	lee una clave de forma segura	config.get('dt')
d.update({...})	actualiza varias entradas	config.update({'dt':0.01})
d.pop(k)	elimina y devuelve una entrada	config.pop('dt')
d.clear()	vacia el diccionario	config.clear()
len(d)	numero de entradas	len(config)
k in d	comprueba si existe una clave	'dt' in config

6. Recorrer un diccionario

```
for key in config:
    print(key)

for value in config.values():
    print(value)

for key, value in config.items():
    print(key, value)
```

El patron `for key, value in d.items()` aparece constantemente en Python porque permite trabajar simultaneamente con el nombre del dato y con su valor.

7. Diccionarios anidados

```
simulation = {
    'initial_conditions': {
        'h0': 0.0,
        'v0': 50.0
    },
    'solver': {
        'method': 'Euler',
        'dt': 0.1
    }
}

simulation['solver']['dt']   # 0.1
```

La indexacion anidada funciona igual que con listas: primero accedes al objeto exterior y luego al interior.

8. Cuando suele usarse un diccionario

Situacion	Por que dict encaja bien
Configuraciones	cada parametro tiene un nombre
Resultados	cada magnitud puede tener una clave descriptiva
Datos de sensores	sensor_id -> medida
Metadatos	nombre, unidades, timestamp, fuente...
JSON / APIs	JSON se parece mucho a diccionarios anidados

Situacion	Por que dict encaja bien
Parametros de funciones complejas	evita largas listas de argumentos posicionales

9. Set: definicion intuitiva

Un **set** es una coleccion de elementos **unicos**. No se usa para asociar nombres a valores, sino para responder preguntas como: 'que elementos tengo?', 'cuales faltan?', 'que elementos comparten dos grupos?'.

```
required = {'h0', 'v0', 't0', 'g', 'dt'}

provided = {'h0', 'v0', 'g', 'dt'}

missing = required - provided
# {'t0'}
```

10. Caracteristicas principales de un set

Caracteristica	Significado
Elementos unicos	los duplicados desaparecen
No indexado	no existe set[0]
Mutable	puedes anadir o eliminar elementos
Muy rapido para membership	ideal para x in my_set
Algebra de conjuntos	union, interseccion, diferencia, etc.
Sin pares clave:valor	un set contiene elementos, no asociaciones

11. Crear sets y eliminar duplicados

```
values = [1, 1, 2, 2, 3, 3]
unique_values = set(values)

print(unique_values)
# {1, 2, 3}
```

Convertir una lista a set es una forma comun de quedarse con los valores unicos.

12. Metodos principales de set

Metodo	Que hace	Ejemplo
.add(x)	anade un elemento	A.add('dt')
.remove(x)	elimina; error si no existe	A.remove('dt')
.discard(x)	elimina sin error si falta	A.discard('dt')
.pop()	elimina un elemento arbitrario	A.pop()
.clear()	vacia el set	A.clear()
.copy()	copia superficial	B = A.copy()
len(A)	numero de elementos	len(A)
x in A	pertenencia	'dt' in A

13. Operaciones de conjuntos

Operacion	Sintaxis	Significado
Union	A | B	todos los elementos presentes en A o B

Operacion	Sintaxis	Significado
Interseccion	A & B	elementos comunes
Diferencia	A - B	elementos de A que no estan en B
Diferencia simetrica	A ^ B	elementos presentes en uno pero no en ambos
Subconjunto	A <= B	todo A esta contenido en B
Superconjunto	A >= B	A contiene todo B

14. Ejemplo practico: validar una configuracion

```
required = {'h0', 'v0', 't0', 'g', 'dt'}
provided = set(config.keys())

missing = required - provided

if missing:
    print('Missing keys:', missing)
else:
    print('Configuration complete')
```

Este es uno de los usos mas naturales de un set: no importa el orden, solo si cada elemento requerido esta presente o no.

15. Diccionario vs set: diferencia fundamental

Pregunta	Diccionario	Set
Que guarda?	pares clave:valor	elementos unicos
Como accedo?	por clave	no por indice; membership
Hay valores asociados?	si	no
Duplicados?	claves no; valores si	no
Uso tipico	datos con nombre	pertenencia / comparacion de grupos
Ejemplo	{'g':9.81}	{'h0','v0','g'}

16. Dict, set, list o tuple?

Necesidad	Estructura recomendada
Secuencia ordenada que cambia	list
Estado fijo ordenado	tuple
Valores con nombres / claves	dict
Coleccion de elementos unicos / pertenencia	set

17. Errores tipicos

```
# 1. Access missing dictionary key
config['unknown']      # KeyError

# 2. Trying to index a set
A[0]                   # TypeError

# 3. Empty set
{}                     # THIS IS AN EMPTY DICT
set()                  # empty set

# 4. Duplicate dict key
{'g': 9.81, 'g': 1.62}
```

```
# final value for 'g' is 1.62
```

Muy importante: `{}` crea un diccionario vacio, no un set. Para un set vacio debes escribir `set()`.

18. Mini cheat sheet

```
DICT
{'key': value}          create
d['key']                read
d.get('key')            safe read
d['key'] = value        add/update
d.keys()                keys
d.values()              values
d.items()               key/value pairs
'key' in d              membership

SET
{1, 2, 3}               create
set()                   empty set
A.add(x)                add
A.remove(x)             remove
A | B                   union
A & B                   intersection
A - B                   difference
x in A                  membership
```